

B9 机械臂 ROS 2 接口文档

适用工作区: Dragon_b9_Right (右臂, 真机验证版) / Dragon_b9_Left (左臂) **接口定义包:** b9_interfaces (3 Action + 2 Service + 1 Message) **运行环境:** ROS 2 Humble **通信层:** DDS (同机/同域直连, 无需 Web 服务器) **文档特色:** 每个接口均给出三种调用方式 —— ① 现成 CLI 工具 (ros2 run) ② 原生 ros2 命令 ③ rclpy 代码

目录

- 1. 概述
 - 1.1 简介
 - 1.2 接口总览
 - 1.3 左右臂工作区对照
 - 1.4 快速开始三连
- 2. 通用约定
 - 2.1 前置条件
 - 2.2 坐标系与单位
 - 2.3 默认运动参数 (传 0 = 服务器默认)
 - 2.4 直线安全工作域
 - 2.5 目标抢占与取消语义
- 3. 现成 CLI 工具总览
 - 3.1 send_xyz_rpy_goal 全参数表
 - 3.2 测试脚本参数表
- 4. Action 接口详解
 - 4.1 /b9/move_end_effector — 末端运动 (自由/直线/圆弧) ★
 - 4.2 /b9/execute_pose_path — 多路点连续轨迹
 - 4.3 /b9/play_trajectory — 示教轨迹回放
- 5. Service 接口详解
 - 5.1 enable — 电机使能
 - 5.2 disable — 电机失能 (软急停)
 - 5.3 set_control_mode — 切换控制模式
 - 5.4 record_trajectory — 示教录制
- 6. Topic 接口详解
- 7. 数据字典
- 8. 接口支持列表
- 9. 命令行示例大全
 - 9.1 系统诊断类
 - 9.2 使能/失能/模式类
 - 9.3 自由运动类
 - 9.4 直线运动大家族 ★
 - 9.5 圆弧运动类
 - 9.6 只规划预览类
 - 9.7 多路点轨迹类
 - 9.8 示教与回放类
 - 9.9 控制器管理类
 - 9.10 验收测试类
 - 9.11 急停与安全收尾类
- 10. 典型业务流程
- 11. 编程参考实现
- 12. 常见问题 FAQ
- 13. 附录

1. 概述

1.1 简介

本文档定义 B9 机械臂在 **ROS 2 图层面的原生控制接口**。相比 HTTP/WebSocket 远控通道:

- 零转发开销:** 直连 action/service/topic, 延迟最低
- 完整语义:** 支持标准取消 (cancel)、反馈流 (--feedback)、目标抢占
- 同域即用:** 同机或同 ROS_DOMAIN_ID 局域网内任何 ROS 2 节点/终端均可调用

- **免代码可用：**工作区自带 CLI 工具，终端一行命令即可驱动直线运动

1.2 接口总览

类型	名称	接口定义	提供节点	功能
Action	/b9/move_end_effector	b9_interfaces/action/MoveEndEffector	end_effector_control_server	末端运动（自由/直线/圆弧）
Action	/b9/execute_pose_path	b9_interfaces/action/ExecutePosePath	fast_pose_path_server	多路点连续轨迹（单次启停）
Action	/b9/play_trajectory	b9_interfaces/action/PlayTrajectory	trajectory_player	示教轨迹回放
Service	/B9RightArmSystem/enable	std_srvs/srv/Trigger	硬件层	电机使能
Service	/B9RightArmSystem/disable	std_srvs/srv/Trigger	硬件层	电机失能（软急停）
Service	/B9RightArmSystem/set_control_mode	b9_interfaces/srv/SetControlMode	硬件层	切换控制模式
Service	/b9_freeteach_controller/record_trajectory	b9_interfaces/srv/RecordTrajectory	示教控制器	录制控制
Topic	/joint_states	sensor_msgs/msg/JointState	joint_state_broadcaster	100Hz 关节状态
Topic	/dynamic_joint_states	control_msgs/msg/DynamicJointState	joint_state_broadcaster	电机诊断量

1.3 左右臂工作区对照

两工作区接口**结构一致，仅命名不同**（互斥运行）。本文以右臂为主体，左臂按下表替换：

项目	右臂	左臂
CLI 臂名参数	right_arm (可简写 right)	left_arm (可简写 left)
Goal.mode	"right_arm"	"left_arm"
目标字段前缀	right_x right_roll ...	left_x left_roll ...
目标启用标志	use_right_target	use_left_target
硬件服务命名空间	/B9RightArmSystem/*	/B9LeftArmSystem/*
轨迹控制器	right_arm_controller	left_arm_controller
直线安全工作域(Y)	-0.28 ~ -0.04 m	+0.04 ~ +0.28 m

1.4 快速开始三连

```
source ~/Dragon_b9_Right/install/setup.bash

# ① 使能电机
ros2 service call /B9RightArmSystem/enable std_srvs/srv/Trigger

# ② 一条直线运动 (CLI 工具, 最常用形态)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.48 -3.141 -1.552 3.141 --ca

# ③ 查看关节状态
ros2 topic echo /joint_states --once
```

2. 通用约定

2.1 前置条件

条件	说明
主控制栈已启动	虚拟: <code>ros2 launch b9_moveit_control end_effector_control.launch.py dry_run:=true</code> ; 真机: ... <code>dry_run:=false auto_enable:=true</code>
环境已 source	<code>source ~/Dragon_b9_Right/install/setup.bash</code> (每个新终端都要)
同 ROS 域	跨机调用需相同 <code>ROS_DOMAIN_ID</code> 且网络可组播
回放/示教接口	需全家桶模式启动 (<code>ros2 launch b9_ros2_control b9_real_moveit_bringup.launch.py</code>)

⚠ **auto_enable 语义 (真机第一大坑)**: 主栈以 `auto_enable:=false` 启动为硬件只读模式——即使调用 `enable` 服务, 运动命令帧也不下发 (action 返回成功但实机不动, 日志出现 `auto_enable=false, so real motor command frames are not being sent.`)。正常运行必须 `auto_enable:=true`。

2.2 坐标系与单位

量	约定
参考系	world (原点在地面基座柱底; 基座顶 Z=1.013m)
X / Y / Z	前方为正 / 左正右负 / 离地高度, 单位 m
roll / pitch / yaw	rad, XYZ 固定轴欧拉角
关节角	rad
速度/加速度缩放	无量纲 0.01–1.0 (关节限速百分比)

2.3 默认运动参数 (传 0 = 服务器默认)

Goal 数值字段传 **0.0 表示采用服务器默认值** (出厂默认 = 已验证安全参数), CLI 不带对应选项即等效传 0:

字段 / CLI 选项	传 0 / 缺省时生效值
<code>velocity_scaling --velocity</code>	0.12 (12% 关节限速)
<code>acceleration_scaling --acceleration</code>	0.12
<code>cartesian_eef_step --eef-step</code>	0.025 m
<code>cartesian_min_fraction --min-fraction</code>	0.85
<code>planning_time --planning-time</code>	20 s

2.4 直线安全工作域

固定推荐姿态 `roll=-3.141, pitch=-1.552, yaw=3.141、X=0.275` 平面 (经 324 点采样验证):

工作区	Y	Z
右臂	-0.28 ~ -0.04	0.44 ~ 0.52
左臂	+0.04 ~ +0.28	0.44 ~ 0.52

越界后果：直线 IK 成功率不足 → **自动回退自由规划** (Result.message 含 OMPL，路径非直线) 或失败。

2.5 目标抢占与取消语义

操作	行为
新 Goal 到达 (move_end_effector)	抢占 : 旧 goal 被 abort, 新 goal 立即执行
标准 action cancel (Ctrl+C CLI / cancel_goal_async)	服务器平滑停止 → Result: error_code=4 CANCELED
execute_pose_path / play_trajectory 忙时新 Goal	拒绝 (busy), 需等当前完成
急停	disable 服务 (🔥 掉臂) 或物理断电, 优先于一切

3. 现成 CLI 工具总览

3.1 send_xyz_rpy_goal 全参数表

/b9/move_end_effector 的全功能命令行客户端 (C++, 编译产物随工作区安装) :

```
ros2 run b9_moveit_control_examples send_xyz_rpy_goal <arm> <x> <y> <z> [roll pitch yaw] [选项...]
```

位置参数

参数	说明
arm	right_arm / left_arm (可简写 right / left)
x y z	目标位置 (m)。只给 3 个数 = 自动 position_only 模式 (不约束姿态)
roll pitch yaw	目标姿态 (rad)。给满 6 个数 = 完整位姿模式

选项

选项	取值	对应 Goal 字段	说明
--cartesian-linear	开关	cartesian_linear	末端走直线
--cartesian-arc	开关	cartesian_arc	末端走圆弧
--arc-cx --arc-cy --arc-cz	float	arc_center_x/y/z	圆弧圆心 (不给=自动计算)
--plan-only	开关	execute=false	只规划不执行 (预览)
--execute	开关	execute=true	显式执行 (默认已是执行)
--position-only	开关	position_only	只约束位置
--velocity	float	velocity_scaling	速度缩放, 缺省→0.12
--acceleration	float	acceleration_scaling	加速度缩放, 缺省→0.12
--eef-step	float	cartesian_eef_step	直线步长 m, 缺省→0.025
--min-fraction	float	cartesian_min_fraction	直线最低 IK 成功率, 缺省→0.85
--jump-threshold	float	cartesian_jump_threshold	关节跳变阈值 (预留)
--planning-time	float	planning_time	回退规划时限 s, 缺省→20
--help / -h	—	—	打印用法

退出码: 0 成功; 1 action 服务器不可用/运动失败; 2 参数错误。

3.2 测试脚本参数表

脚本	调用形式	参数
full_linear_test.py	python3 src/b9_moveit_control_examples/scripts/full_linear_test.py right_arm [--vel 0.12] [--step 0.025]	19 步直线网格验收; --vel 速度、--step 直线步长

test_cartesian_linear.py	python3 ../test_cartesian_linear.py right_arm [--x --y --z --roll --pitch --yaw] [--cartesian-linear] [--step] [--position-only] [--plan-only]	单条直线细测
send_pose_path.py	python3 ../send_pose_path.py right_arm --file wp.json --execute [--plan-only] [--velocity 0.3]	/b9/execute_pose_path 客户端。 路点必须用 --file JSON 提供 （命令行平铺负数坐标会触发 argparse 解析错误，实测不可用）
stress_test_arm.py	python3 ../stress_test_arm.py --arm right_arm	工业场景压测

4. Action 接口详解

每个接口按：接口定义表 → Goal/Result/Feedback 字段表 → 调用方式①②③ → 注意事项。

4.1 /b9/move_end_effector — 末端运动（自由/直线/圆弧）★

项	值
接口类型	b9_interfaces/action/MoveEndEffector
提供节点	end_effector_control_server
描述	单目标末端运动。自由（默认）/ 直线 / 圆弧三种路径；支持预览、反馈流、抢占与取消

Goal 字段

字段	类型	必填	默认	说明
mode	string	是	—	right_arm
use_right_target	bool	是	false	必须置 true（左臂用 use_left_target）
right_x / right_y / right_z	float64	是	—	目标位置 (m)
right_roll / right_pitch / right_yaw	float64	是*	—	目标姿态 (rad)；position_only 时忽略
cartesian_linear	bool	否	false	true = 直线
cartesian_arc	bool	否	false	true = 圆弧（与直线互斥）
arc_center_x / y / z	float64	否	0	圆心；全 0 自动计算
velocity_scaling	float64	否	0(→0.12)	[0.01,1.0]；真机建议 ≤0.3
acceleration_scaling	float64	否	0(→0.12)	
cartesian_eef_step	float64	否	0(→0.025)	直线步长 (0,0.1] m
cartesian_jump_threshold	float64	否	0(→2.0)	预留
cartesian_min_fraction	float64	否	0(→0.85)	低于则回退自由规划
planning_time	float64	否	0(→20)	回退规划时限 s
position_only	bool	否	false	只约束位置
execute	bool	是	false	true=执行；false=只规划

Result 字段

字段	类型	说明
success	bool	
error_code	int32	\$7.4
message	string	路径语义判据 \$7.6（含 OMPL 即未走直线）

Feedback 字段

字段	类型	说明
stage	string	\$7.5 状态机
progress	float64	[0,1]

message	string	阶段描述
---------	--------	------

调用方式① — CLI 工具（推荐日常使用）

```
# 直线运动（默认安全参数：12%速度/2.5cm步长/85%成功率）
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.48 -3.141 -1.552 3.141 --ca

# 自由路径（无直线约束）
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48 -3.141 -1.552 3.141

# 仅位置目标（3 个数自动 position_only，姿态由求解器自由选择）
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48

# 高速高精度直线（30%速度 + 1cm 步长）
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.04 0.52 -3.141 -1.552 3.141 \
  --cartesian-linear --velocity 0.3 --acceleration 0.2 --eef-step 0.01

# 极低速首移（真机首次测试推荐）
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48 -3.141 -1.552 3.141 \
  --velocity 0.05 --acceleration 0.05

# 放宽直线成立门槛（容忍 70% IK 成功率）
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.52 -3.141 -1.552 3.141 \
  --cartesian-linear --min-fraction 0.7
```

调用方式② — 原生 ros2 action 命令

```
# 直线 + 实时反馈流
ros2 action send_goal /b9/move_end_effector b9_interfaces/action/MoveEndEffector \
  "{mode: right_arm, use_right_target: true, execute: true, cartesian_linear: true,
  right_x: 0.275, right_y: -0.28, right_z: 0.48,
  right_roll: -3.141, right_pitch: -1.552, right_yaw: 3.141}" --feedback

# 自定义全部参数
ros2 action send_goal /b9/move_end_effector b9_interfaces/action/MoveEndEffector \
  "{mode: right_arm, use_right_target: true, execute: true, cartesian_linear: true,
  right_x: 0.275, right_y: -0.04, right_z: 0.52,
  right_roll: -3.141, right_pitch: -1.552, right_yaw: 3.141,
  velocity_scaling: 0.3, acceleration_scaling: 0.2,
  cartesian_eef_step: 0.01, cartesian_min_fraction: 0.9, planning_time: 15.0}"

# 查看接口结构
ros2 interface show b9_interfaces/action/MoveEndEffector
ros2 action info /b9/move_end_effector
```

调用方式③ — rclpy

```
import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
from b9_interfaces.action import MoveEndEffector

class EEClient(Node):
    def __init__(self):
        super().__init__("ee_client")
        self.cli = ActionClient(self, MoveEndEffector, "/b9/move_end_effector")

    def line_to(self, x, y, z, r=-3.141, p=-1.552, yaw=3.141, vel=0.12):
        g = MoveEndEffector.Goal()
        g.mode, g.use_right_target, g.execute = "right_arm", True, True
        g.cartesian_linear = True
        g.right_x, g.right_y, g.right_z = x, y, z
        g.right_roll, g.right_pitch, g.right_yaw = r, p, yaw
        g.velocity_scaling = g.acceleration_scaling = vel

        self.cli.wait_for_server()
        gh = self.cli.send_goal_async(
            g, feedback_callback=lambda f: print(
                f"{f.feedback.progress*100:5.1f}% {f.feedback.stage}"))
        rclpy.spin_until_future_complete(self, gh)
        res = gh.result().get_result_async()
        rclpy.spin_until_future_complete(self, res)
        return res.result().result

rclpy.init()
r = EEClient().line_to(0.275, -0.28, 0.48)
assert r.success and "OMPL" not in r.message, r.message
```

注意事项

- 1. 方式②忘记 use_right_target: true 会被直接拒绝
- 2. message 含 OMPL = 已回退非直线, 应视为业务告警
- 3. 新 goal 抢占旧 goal; CLI 中 Ctrl+C 触发标准 cancel (平滑停止)

4.2 /b9/execute_pose_path — 多路点连续轨迹

项	值
接口类型	b9_interfaces/action/ExecutePosePath
提供节点	fast_pose_path_server
描述	多路点序列: 逐点 IK+规划 → 分段拼接 → 整体重参数化 → 一次启停连续执行 (路点间不停顿)。适合流水线轨迹

Goal 字段

字段	类型	必填	默认	说明
mode	string	是	—	right_arm
waypoints	PosePoint[]	是	—	路点序列 (\$7.1) , ≥1
velocity_scaling	float64	否	0(→0.12)	
acceleration_scaling	float64	否	0(→0.12)	
planning_time	float64	否	0(→20)	每段规划时限
execute	bool	是	false	false=只规划统计

Result: success / completed_count(int32 成功路点数) / message **Feedback:** current_index / stage(ik+planning→executing→done) / progress

调用方式① — CLI 脚本

```
# 路点用 JSON 文件提供 (✅ 实测验证形式)
cat > /tmp/wp.json << 'EOF'
[{"x":0.275,"y":-0.16,"z":0.48,"roll":-3.141,"pitch":-1.552,"yaw":3.141},
 {"x":0.275,"y":-0.28,"z":0.48,"roll":-3.141,"pitch":-1.552,"yaw":3.141},
 {"x":0.275,"y":-0.04,"z":0.52,"roll":-3.141,"pitch":-1.552,"yaw":3.141}]
EOF
python3 src/b9_moveit_control_examples/scripts/send_pose_path.py right_arm --file /tmp/wp.json --execute

# 只规划评估可行性
python3 src/b9_moveit_control_examples/scripts/send_pose_path.py right_arm --file /tmp/wp.json --plan-onl
```

⚠️ 该脚本的命令行平铺路点形式 (直接跟 x y z ...) 因 argparse 无法解析负数坐标而**不可用**, 请一律使用 --file JSON。

调用方式② — 原生 ros2 action 命令

```
ros2 action send_goal /b9/execute_pose_path b9_interfaces/action/ExecutePosePath \
  "{mode: right_arm, execute: true, velocity_scaling: 0.15,
   waypoints: [
     {x: 0.275, y: -0.16, z: 0.48, roll: -3.141, pitch: -1.552, yaw: 3.141},
     {x: 0.275, y: -0.28, z: 0.48, roll: -3.141, pitch: -1.552, yaw: 3.141},
     {x: 0.275, y: -0.28, z: 0.52, roll: -3.141, pitch: -1.552, yaw: 3.141}]}\" --feedback
```

注意事项

1. 服务器忙时新 goal 被**拒绝** (非抢占)
2. 路点间连接**不保证直线**; 严格直线请逐段调用 4.1
3. 全程仅一次加减速, 比逐点调用 4.1 平滑高效

4.3 /b9/play_trajectory — 示教轨迹回放

项	值
接口类型	b9_interfaces/action/PlayTrajectory
提供节点	trajectory_player (需全家桶/回放模式启动)
描述	加载示教 CSV/YAML → 高斯平滑(100ms) → 冻结静止关节 → 抽稀(≤200点) → 自碰撞检查(>50%中止) → 直发关节轨迹控制器。支持变速/循环

Goal 字段

字段	类型	必填	默认	说明
trajectory_id	string	是	—	文件名 (自动拼 ~/trajectories/) 或绝对路径; 无扩展名默认 .csv
speed_scale	float64	否	1.0	速度倍率 >0
loop_count	int32	否	1	循环次数 (≤0 视为 1)

Result: success / error_code / message **Feedback:** stage(smoothing→validating→ready→executing→resetting→done) / progress / message(含预估时长)

调用方式② — 原生 ros2 action 命令

```
# 标准回放
ros2 action send_goal /b9/play_trajectory b9_interfaces/action/PlayTrajectory \
  "{trajectory_id: 'demo.csv', speed_scale: 1.0, loop_count: 1}" --feedback

# 0.5 倍慢速回放 3 遍
ros2 action send_goal /b9/play_trajectory b9_interfaces/action/PlayTrajectory \
  "{trajectory_id: 'demo.csv', speed_scale: 0.5, loop_count: 3}"

# 绝对路径轨迹
ros2 action send_goal /b9/play_trajectory b9_interfaces/action/PlayTrajectory \
  "{trajectory_id: '/home/xia17/trajectories/task_A.yaml', speed_scale: 1.0, loop_count: 1}"
```

5. Service 接口详解

5.1 enable — 电机使能

项	值
服务名	/B9RightArmSystem/enable
类型	std_srvs/srv/Trigger
描述	全部电机上力锁住当前姿态

Request: 空 **Response:** success (bool) / message (string)

```
ros2 service call /B9RightArmSystem/enable std_srvs/srv/Trigger
# 左臂工作区:
ros2 service call /B9LeftArmSystem/enable std_srvs/srv/Trigger
```

注意: 主栈 auto_enable=false 时使能≠可运动 (\$2.1) 。

5.2 disable — 电机失能（软急停）

项	值
服务名	/B9RightArmSystem/disable
类型	std_srvs/srv/Trigger
⚠ 警告	失能瞬间手臂因重力下坠，先到低位或人工扶持

```
ros2 service call /B9RightArmSystem/disable std_srvs/srv/Trigger
```

5.3 set_control_mode — 切换控制模式

项	值
服务名	/B9RightArmSystem/set_control_mode
类型	b9_interfaces/srv/SetControlMode
描述	切换电机底层模式 (\$7.3) ；切换需 0.5s 内硬件确认回读

Request: mode (string) **Response:** success / message

```
ros2 service call /B9RightArmSystem/set_control_mode b9_interfaces/srv/SetControlMode "{mode: 'pos_vel'}"
ros2 service call /B9RightArmSystem/set_control_mode b9_interfaces/srv/SetControlMode "{mode: 'mit'}"
ros2 service call /B9RightArmSystem/set_control_mode b9_interfaces/srv/SetControlMode "{mode: 'pos_vel_cs"
```

5.4 record_trajectory — 示教录制

项	值
服务名	/b9_freeteach_controller/record_trajectory
类型	b9_interfaces/srv/RecordTrajectory
前置	示教控制器已激活（见 §9.8 进入示教三步曲）

Request

字段	类型	说明
command	string	start / stop / save / cancel
filename	string	save 时目标文件 (.csv/.yaml; 相对名拼默认目录)
enable_gravity_comp	bool	录制时重力补偿

Response: success / message / trajectory_id（可作 4.3 入参）

```
ros2 service call /b9_freeteach_controller/record_trajectory \
  b9_interfaces/srv/RecordTrajectory "{command: 'start', enable_gravity_comp: true}"
ros2 service call /b9_freeteach_controller/record_trajectory \
  b9_interfaces/srv/RecordTrajectory "{command: 'save', filename: 'demo.csv'}"
```

6. Topic 接口详解

6.1 /joint_states — 关节状态流

项	值
类型	sensor_msgs/msg/JointState, 100 Hz
内容	name[7] / position[7] / velocity[7] / effort[7]

```
ros2 topic echo /joint_states --once # 单帧
ros2 topic hz /joint_states # 验证 100Hz
ros2 topic echo /joint_states --field position --once # 只看位置数组
```

6.2 /dynamic_joint_states — 电机诊断流

项	值
类型	control_msgs/msg/DynamicJointState
内容	每关节: position/velocity/effort + motor_error / fault / has_feedback / feedback_age / enabled

诊断接口	健康判据
motor_error	0 ($\geq 0x08$ 故障码)
fault	0 (1=锁存需断电复位)
has_feedback	1
feedback_age	< 0.1 s
enabled	运动前必须 1

```
ros2 topic echo /dynamic_joint_states --once
```

7. 数据字典

7.1 msg/PosePoint

```
float64 x # m, world 系
float64 y
float64 z
float64 roll # rad
float64 pitch
float64 yaw
```

7.2 关节顺序与限位

索引	关节名 (前缀 right_/left_)	右臂限位(rad)	左臂限位(rad)
0	shoulder_pitch_joint	± 2.0	± 2.0
1	shoulder_roll_joint	$-0.2 \sim 2.0$	$-0.2 \sim 2.0$
2	shoulder_yaw_joint	± 2.0	± 1.2
3	elbow_roll_joint	± 1.2	± 1.2
4	elbow_yaw_joint	± 2.0	± 2.0

5	wrist_pitch_joint	±1.1	±1.1
6	wrist_yaw_joint	±1.1	±1.1

全零位 = 手臂自然下垂。

7.3 CtrlMode 枚举

值	含义	用途
pos_vel	位置+速度限幅	常规运动（默认）
mit	力矩/阻抗	拖动示教
pos_vel_csp	周期同步位置	高频轨迹流

7.4 ErrorCode 枚举

码	名称	说明	处置
0	NONE	成功	—
1	INVALID_GOAL	mode 错 / use_*_target 缺 / NaN / 参数越界	对照 Goal 表
2	PLANNING_FAILED	规划失败	目标不可达，查 §2.4
3	EXECUTION_FAILED	执行失败	查 /dynamic_joint_states
4	CANCELED	被取消/抢占	预期行为
5	IK_FAILED / COLLISION	逆解失败（回放中=自碰撞中止）	同 2 / 重录轨迹
6	EXCEPTION	内部异常	查服务器日志

7.5 stage 状态机

```
move_end_effector:
  validating(5%) → planning|cartesian_planning|arc_planning(25%)
    → [ompl_fallback(50%)] → executing(70%) → done|failed(100%)
execute_pose_path:
  ik+planning(逐点) → executing(95%) → done
play_trajectory:
  smoothing → validating → ready → executing(按循环) → resetting → done
```

7.6 message 语义速查

message 关键词	含义
Cartesian execution finished	✅ 直线成功
Arc execution finished	✅ 圆弧成功
Execution finished (exact pose)	自由路径成功
OMPL execution finished	⚠️ 已回退自由路径（非直线）
Planning succeeded ...	execute=false 预览成功
All planning strategies failed	❌ 不可达
... requires use_right_target=true	Goal 缺启用标志
Only right_arm mode is supported ...	mode 传了别的臂

8. 接口支持列表

interface	类型	CLI 工具	ros2 原生命令	右臂	左臂	需全家桶
/b9/move_end_effector	Action	√ send_xyz_rpy_goal	√	√	√	
/b9/execute_pose_path	Action	√ send_pose_path.py	√	√	√	
/b9/play_trajectory	Action		√	√	√	√

enable / disable	Service	√	√	√	
set_control_mode	Service	√	√	√	
record_trajectory	Service	√	√	√	√
/joint_states	Topic	√	√	√	
/dynamic_joint_states	Topic	√	√	√	

9. 命令行示例大全

全部命令以右臂为例，可直接复制执行。前置：source ~/Dragon_b9_Right/install/setup.bash。

9.1 系统诊断类

```
ros2 node list                # 应见 move_group / end_effector_control_server 等
ros2 action list | grep b9    # 应见 /b9/move_end_effector /b9/execute_pose_path
ros2 service list | grep B9   # 应见 enable/disable/set_control_mode
ros2 control list_controllers # right_arm_controller 应为 active
ros2 topic hz /joint_states   # 应 ≈100Hz
ros2 topic echo /joint_states --once # 关节位置快照
ros2 topic echo /dynamic_joint_states --once # 电机错误码/反馈龄/使能位
ros2 interface show b9_interfaces/action/MoveEndEffector # 查看接口定义
ros2 daemon stop              # 发现异常/幽灵节点时清缓存
```

9.2 使能/失能/模式类

```
ros2 service call /B9RightArmSystem/enable std_srvs/srv/Trigger # 上力
ros2 service call /B9RightArmSystem/disable std_srvs/srv/Trigger # 失能(⚠️掉臂)
ros2 service call /B9RightArmSystem/set_control_mode b9_interfaces/srv/SetControlMode "{mode: 'pos_vel'}"
ros2 service call /B9RightArmSystem/set_control_mode b9_interfaces/srv/SetControlMode "{mode: 'mit'}"
```

9.3 自由运动类

```
# 完整位姿目标
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48 -3.141 -1.552 3.141

# 仅位置目标(姿态自由)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48

# 低速自由运动
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48 -3.141 -1.552 3.141 --ve
```

9.4 直线运动大家族 ★

```
# 标准直线(默认安全参数)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.48 -3.141 -1.552 3.141 --ca

# 横向直线扫描 (工作域左界→右界, 24cm)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.48 -3.141 -1.552 3.141 --ca
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.04 0.48 -3.141 -1.552 3.141 --ca

# 竖直直线升降
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.52 -3.141 -1.552 3.141 --ca
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.44 -3.141 -1.552 3.141 --ca

# 空间对角直线
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.04 0.52 -3.141 -1.552 3.141 --ca

# 高速直线(30%)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.48 -3.141 -1.552 3.141 --ca

# 高精度直线(1cm 步长)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.48 -3.141 -1.552 3.141 --ca

# 放宽成立门槛(边界点尝试)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.52 -3.141 -1.552 3.141 --ca

# 原生 action 版直线(带反馈)
ros2 action send_goal /b9/move_end_effector b9_interfaces/action/MoveEndEffector \
  "{mode: right_arm, use_right_target: true, execute: true, cartesian_linear: true,
   right_x: 0.275, right_y: -0.28, right_z: 0.48,
   right_roll: -3.141, right_pitch: -1.552, right_yaw: 3.141}" --feedback
```

9.5 圆弧运动类

```
# 自动圆心圆弧 (可用; 起点距目标较远或临近工作域边缘时求解耗时可能较长, 建议优先指定圆心)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.04 0.48 -3.141 -1.552 3.141 --ca

# 指定圆心圆弧(绕上方圆心画弧)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.28 0.48 -3.141 -1.552 3.141 \
  --cartesian-arc --arc-cx 0.275 --arc-cy -0.16 --arc-cz 0.55
```

9.6 只规划预览类

```
# 直线可行性预览(不动)
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.30 0.40 -3.141 -1.552 3.141 --ca

# 自由路径预览
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48 -3.141 -1.552 3.141 --pl

# 原生 action 预览(execute: false)
ros2 action send_goal /b9/move_end_effector b9_interfaces/action/MoveEndEffector \
  "{mode: right_arm, use_right_target: true, execute: false, cartesian_linear: true,
   right_x: 0.275, right_y: -0.28, right_z: 0.48,
   right_roll: -3.141, right_pitch: -1.552, right_yaw: 3.141}"
```

9.7 多路点轨迹类

```
# CLI 脚本(路点必须用 JSON 文件, 命令行平铺负数坐标不可用)
cat > /tmp/wp.json << 'EOF'
[{"x":0.275,"y":-0.16,"z":0.48,"roll":-3.141,"pitch":-1.552,"yaw":3.141},
 {"x":0.275,"y":-0.28,"z":0.48,"roll":-3.141,"pitch":-1.552,"yaw":3.141},
 {"x":0.275,"y":-0.28,"z":0.52,"roll":-3.141,"pitch":-1.552,"yaw":3.141}]
EOF
python3 src/b9_moveit_control_examples/scripts/send_pose_path.py right_arm --file /tmp/wp.json --execute

# 原生 action 版
ros2 action send_goal /b9/execute_pose_path b9_interfaces/action/ExecutePosePath \
  "{mode: right_arm, execute: true, velocity_scaling: 0.15, waypoints: [
   {x: 0.275, y: -0.16, z: 0.48, roll: -3.141, pitch: -1.552, yaw: 3.141},
   {x: 0.275, y: -0.28, z: 0.48, roll: -3.141, pitch: -1.552, yaw: 3.141}]}" --feedback
```

9.8 示教与回放类

```
# — 进入示教三步曲 —
ros2 control switch_controllers --deactivate right_arm_controller
ros2 service call /B9RightArmSystem/set_control_mode b9_interfaces/srv/SetControlMode "{mode: 'mit'}"
ros2 control switch_controllers --activate b9_freeteach_controller

# 开始/保存录制
ros2 service call /b9_freeteach_controller/record_trajectory \
  b9_interfaces/srv/RecordTrajectory "{command: 'start', enable_gravity_comp: true}"
# ..... 手拖示教 .....
ros2 service call /b9_freeteach_controller/record_trajectory \
  b9_interfaces/srv/RecordTrajectory "{command: 'save', filename: 'demo.csv'}"

# — 退出示教三步曲 —
ros2 control switch_controllers --deactivate b9_freeteach_controller
ros2 service call /B9RightArmSystem/set_control_mode b9_interfaces/srv/SetControlMode "{mode: 'pos_vel'}"
ros2 control switch_controllers --activate right_arm_controller

# 回放
ros2 action send_goal /b9/play_trajectory b9_interfaces/action/PlayTrajectory \
  "{trajectory_id: 'demo.csv', speed_scale: 0.8, loop_count: 2}" --feedback
```

9.9 控制器管理类

```
ros2 control list_controllers
ros2 control list_hardware_interfaces
ros2 control switch_controllers --deactivate right_arm_controller
ros2 control switch_controllers --activate right_arm_controller
```

9.10 验收测试类

```
# 先到网格中心
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48 -3.141 -1.552 3.141
# 19 步直线网格验收(横扫/升降/对角线, 期望 19/19)
python3 src/b9_moveit_control_examples/scripts/full_linear_test.py right_arm --vel 0.08
# 单条直线细测
python3 src/b9_moveit_control_examples/scripts/test_cartesian_linear.py right_arm --x 0.275 --y -0.28 --z
# 工作空间可达性采样(扩网格前必跑)
ros2 launch b9_moveit_control_examples sample_workspace.launch.py \
  mode:=right_arm x_min:=0.275 x_max:=0.276 y_min:=-0.34 y_max:=0.0 \
  z_min:=0.34 z_max:=0.68 step:=0.02 \
  roll:=-3.141 pitch:=-1.552 yaw:=3.141 output_csv:=tmp/ws.csv
```

9.11 急停与安全收尾类

```
# 软急停(⚠️掉臂, 先扶住)
ros2 service call /B9RightArmSystem/disable std_srvs/srv/Trigger

# 标准收尾: 低速直线降低位 → 扶住 → 失能
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.44 -3.141 -1.552 3.141 --ca
ros2 service call /B9RightArmSystem/disable std_srvs/srv/Trigger

# 清理残留进程(切换工作区前)
killall -9 rviz2 move_group ros2_control_node robot_state_publisher end_effector_control_server fast_pose
```

10. 典型业务流程

10.1 安全开机与首次运动

```
ros2 topic echo /dynamic_joint_states --once # ③ 反馈/故障检查
ros2 service call /B9RightArmSystem/enable std_srvs/srv/Trigger # ② 上力
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48 -3.141 -1.552 3.141 --pl
ros2 run b9_moveit_control_examples send_xyz_rpy_goal right_arm 0.275 -0.16 0.48 -3.141 -1.552 3.141 --ve
```

10.2 直线流水线（逐段严格直线）

对每个途经点串行执行 \$9.4 标准直线命令，确认输出 Cartesian execution finished（而非 OMPL）再发下一段。

10.3 连续平滑多点作业

一次性把全部路点交给 \$9.7（单次启停连续轨迹，允许非直线连接）。

10.4 示教 → 回放

\$9.8 完整命令序列。

10.5 安全收尾

\$9.11 标准收尾序列 → Ctrl+C 关主栈（自动再失能）→ 断电。

11. 编程参考实现

11.1 rclpy 完整客户端

见 \$4.1 调用方式③（可直接运行）。多段直线循环调用 line_to 并校验 "OMPL" not in message。

11.2 rclcpp (C++) 骨架

```
#include <rclcpp/rclcpp.hpp>
#include <rclcpp_action/rclcpp_action.hpp>
#include <b9_interfaces/action/move_end_effector.hpp>
using MoveEE = b9_interfaces::action::MoveEndEffector;

auto node = std::make_shared<rclcpp::Node>("ee_cpp_client");
auto cli = rclcpp_action::create_client<MoveEE>(node, "/b9/move_end_effector");
cli->wait_for_action_server();

MoveEE::Goal g;
g.mode = "right_arm"; g.use_right_target = true; g.execute = true;
g.cartesian_linear = true;
g.right_x = 0.275; g.right_y = -0.28; g.right_z = 0.48;
g.right_roll = -3.141; g.right_pitch = -1.552; g.right_yaw = 3.141;

auto gh_fut = cli->async_send_goal(g);
rclcpp::spin_until_future_complete(node, gh_fut);
auto res_fut = cli->async_get_result(gh_fut.get());
rclcpp::spin_until_future_complete(node, res_fut);
auto result = res_fut.get().result; // ->success / ->error_code / ->message
```

12. 常见问题 FAQ

Q1: action 返回 success 但实机不动？ A: 主栈 auto_enable:=false 只读模式（\$2.1），或电机未使能（查 \$9.1 诊断命令的 enabled）。

Q2: goal 被立即 REJECT？ A: ① mode 与工作区不符；② 原生命令忘记 use_right_target: true；③ execute_pose_path / play_trajectory 正忙。

Q3: 如何中途停止？ A: CLI 前台 Ctrl+C（标准 cancel 平滑停）；或发新 goal 抢占（仅 4.1）；急停用 disable。

Q4: CLI 提示 Unknown option: --step？ A: send_xyz_rpy_goal 的步长选项是 --eef-step（--step 是 full_linear_test.py 的参数，两者勿混）。

Q5: 为什么传 0 速度却得到 12%？ A: 设计如此——数值 0 = “采用服务器默认值”（\$2.3）。要极慢请传 ≥0.01 的实际值。

Q6: 跨机器发现不了服务? A: 同 ROS_DOMAIN_ID、网络允许 DDS 组播; ros2 daemon stop 清缓存重试。

Q7: 如何判断真走了直线? A: Result.message 含 Cartesian execution finished 即直线; 含 OMPL 即回退 (\$7.6) 。

Q8: feedback 收不到? A: 原生命令加 --feedback; rclpy 传 feedback_callback; CLI 工具默认打印反馈。

13. 附录

13.1 主栈启动方式速查

场景	命令
虚拟 + RViz	ros2 launch b9_moveit_control end_effector_control.launch.py dry_run:=true
真机标准	ros2 launch b9_moveit_control end_effector_control.launch.py dry_run:=false auto_enable:=true
真机只读监测+RViz	ros2 launch b9_ros2_control b9_real_visualize.launch.py
真机全家桶 (回放/示教/Web)	ros2 launch b9_ros2_control b9_real_moveit_bringup.launch.py
示教回放专用	ros2 launch b9_moveit_control playback.launch.py dry_run:=false

13.2 接口定义文件位置

```
src/b9_interfaces/  
├─ action/MoveEndEffector.action  
├─ action/ExecutePosePath.action  
├─ action/PlayTrajectory.action  
├─ srv/SetControlMode.srv  
├─ srv/RecordTrajectory.srv  
└─ msg/PosePoint.msg
```

13.3 术语表

术语	释义
抢占 (preempt)	新目标使旧目标提前以 abort 结束
回退 (OMPL fallback)	直线不成立时自动改用自由规划
只读模式	auto_enable= false, 硬件不接收运动帧
全家桶	b9_real_moveit_bringup 启动 (主栈+回放+示教+Web)
传 0 语义	Goal 数值字段 0 = 采用服务器默认安全值

文档结束。接口行为以 b9_interfaces 定义与各服务器源码为最终依据; 发现不一致请反馈修订。